STT / TTS Research

ASR/STT and TTS pipelines using NVIDIA NeMo for an in-house multilingual conversational AI system

1. Overall Objective

Speech → ASR/STT → Tokenizer → SLM → Detokenizer → TTS → Speech

The current project has a working local Hindi speech-to-speech prototype. The current STT and TTS models are open-source baselines. The next research direction is to investigate NVIDIA NeMo for project-specific ASR/STT and TTS development, including fine-tuning, evaluation and integration.

2. ASR / STT Development Approach

Step	Approach
1. Dataset	Collect/prepare licensed Hindi and Hinglish speech with accurate transcripts.
2. Cleaning	Remove noisy/unusable recordings and standardize audio quality, sampling rate and transcripts.
3. Normalization	Normalize Hindi/Hinglish transcripts and prepare language-specific text processing where required.
4. NeMo format	Convert audio and transcripts into a NeMo-compatible ASR manifest/dataset.
5. Model	Investigate NVIDIA NeMo's **Nemotron-3.5-ASR-Streaming** family for low-latency multilingual ASR. The model family uses a cache-aware streaming FastConformer architecture and supports 40 languages.
6. Training / Fine-tuning	Fine-tune with project-specific Hindi/Hinglish speech data to improve domain and conversational performance.
7. Evaluation	Measure WER/CER, transcription accuracy, latency and performance on Hindi/Hinglish/code-mixed speech.
8. Integration	Connect ASR text output to the project's tokenizer and SLM.

Model accuracy note: NVIDIA's current NeMo documentation identifies Nemotron-3.5-ASR-Streaming as a real-time streaming ASR family supporting 40 languages. The exact Hindi/Hinglish checkpoint/configuration should be verified before training or deployment; the document does not assume that every checkpoint has identical language coverage.

3. ASR / STT Pipeline Architecture

	Microphone / Audio Input
↓	Audio preprocessing / resampling
↓	Cache-aware streaming FastConformer ASR
↓	RNNT / streaming decoder
↓	Hindi / Hinglish transcript
↓	Text normalization
↓	Custom tokenizer
↓	SLM

4. TTS Development Approach

Step	Approach
1. Dataset	Collect/prepare licensed Hindi and Hinglish speech with accurate transcripts.
2. Cleaning	Remove unusable recordings and standardize audio quality and transcripts.
3. Normalization	Normalize text and prepare language-specific text processing where required.
4. NeMo format	Convert audio and transcripts into a NeMo-compatible dataset/manifest.
5. Model	Investigate **Magpie-TTS** / NVIDIA NeMo as the starting multilingual TTS model and framework.

6. Training	Fine-tune a pretrained Magpie-TTS checkpoint or, if required, investigate training with project-specific data.
7. Evaluation	Measure intelligibility, pronunciation, naturalness, speaker consistency and suitability for Hindi/Hinglish.
8. Integration	Connect the resulting TTS to the SLM and complete the speech-to-speech pipeline.

5. TTS Pipeline Architecture — Magpie-TTS

	Generated response text from SLM
↓	Text / language-specific processing
↓	Magpie-TTS Transformer
↓	Discrete audio-code prediction
↓	Neural audio codec
↓	Speech waveform
↓	Speaker

6. Complete Speech-to-Speech Architecture

User / Microphone	Input speech
ASR / STT	NVIDIA NeMo ASR → transcript
Tokenizer	Text → token IDs
SLM	Token IDs → generated response
Detokenizer	Token IDs → response text
TTS	NVIDIA NeMo / Magpie-TTS → speech waveform
Speaker	Output speech

7. Current Project Status

STT	Hindi STT working locally with AI4Bharat IndicWav2Vec as the current baseline.
TTS	Hindi TTS working locally with Facebook MMS as the current baseline.
SLM	Custom multilingual conversational SLM prototype.
Integration	Local Speech → STT → Tokenizer → SLM → TTS → Speech pipeline has been tested successfully.
Next research	Investigate NeMo ASR and Magpie-TTS for project-specific speech model development.

8. Key Conclusion

NeMo provides a practical framework for custom ASR and TTS development. For ASR, the Nemotron-3.5-ASR-Streaming family is a relevant direction for low-latency multilingual speech recognition, using cache-aware streaming FastConformer technology. For TTS, Magpie-TTS is NVIDIA NeMo's multilingual codec-based TTS direction and provides a path for fine-tuning to new speakers or languages. The existing open-source STT/TTS models can remain as baselines while project-specific speech components are developed and evaluated.

9. References

• NVIDIA NeMo Speech — Choosing a Model: https://docs.nvidia.com/nemo/speech/nightly/starthere/choosing_a_model.html

• NVIDIA NeMo ASR Featured Models: https://docs.nvidia.com/nemo/speech/nightly/asr/featured_models.html

• NVIDIA NeMo TTS Models / MagpieTTS: https://docs.nvidia.com/nemo/speech/nightly/tts/models.html

• NVIDIA NeMo Magpie-TTS Fine-tuning: https://docs.nvidia.com/nemo/speech/nightly/tts/magpietts-finetuning.html

• NVIDIA NeMo Speech Inference: https://docs.nvidia.com/nemo/speech/nightly/starthere/ten_minutes.html